Latest updates: https://dl.acm.org/doi/10.1145/3696410.3714908

RESEARCH-ARTICLE

Collaborative Retrieval for Large Language Model-based Conversational Recommender Systems

YAOCHEN ZHU, University of Virginia, Charlottesville, VA, United States

CHAO WAN, Cornell University, Ithaca, NY, United States

HARALD STECK, Netflix, Inc., Los Gatos, CA, United States

DAWEN LIANG, Netflix, Inc., Los Gatos, CA, United States

YESU FENG, Netflix, Inc., Los Gatos, CA, United States

NATHAN KALLUS, Netflix, Inc., Los Gatos, CA, United States

View all

Open Access Support provided by:

Netflix, Inc.

University of Virginia

Cornell University

PDF Download
3696410.3714908.pdf
05 March 2026
Total Citations: 3
Total Downloads: 5920

Published: 22 April 2025

Citation in BibTeX format

WWW '25: The ACM Web Conference 2025
April 28 - May 2, 2025
Sydney NSW, Australia

Conference Sponsors:
SIGWEB

WWW '25: Proceedings of the ACM on Web Conference 2025 (April 2025)
https://doi.org/10.1145/3696410.3714908
ISBN: 9798400712746

Collaborative Retrieval for Large Language Model-based Conversational Recommender Systems

Yaochen Zhu*
uqp4qh@virginia.edu
University of Virginia
Charlottesville, VA, USA

Chao Wan
cw862@cornell.edu
Cornell University
Ithaca, NY, USA

Harald Steck
hsteck@netflix.com
Netflix Inc.
Los Gatos, CA, USA

Dawen Liang
dliang@netflix.com
Netflix Inc.
Los Gatos, CA, USA

Yesu Feng
yfeng@netflix.com
Netflix Inc.
Los Gatos, CA, USA

Nathan Kallus
nkallus@netflix.com
Netflix Inc. & Cornell University
New York, NY, USA

Jundong Li
jundong@virginia.edu
University of Virginia
Charlottesville, VA, USA

Abstract

Conversational recommender systems (CRS) aim to provide personalized recommendations via interactive dialogues with users. While large language models (LLMs) enhance CRS with their superior understanding of context-aware user preferences, they typically struggle to leverage behavioral data, which have proven to be important for classical collaborative filtering (CF)-based approaches. For this reason, we propose CRAG—**C**ollaborative **R**etrieval **A**ugmented **G**eneration for LLM-based CRS. To the best of our knowledge, CRAG is the first approach that combines state-of-the-art LLMs with CF for conversational recommendations. Our experiments on two publicly available movie conversational recommendation datasets, i.e., a refined Reddit dataset (which we name Reddit-v2) as well as the Redial dataset, demonstrate the superior item coverage and recommendation performance of CRAG, compared to several CRS baselines. Moreover, we observe that the improvements are mainly due to better recommendation accuracy on recently released movies. The code and data are available at https://github.com/yaochenzhu/CRAG.

CCS Concepts

• **Information systems** → **Recommender systems**.

Keywords

Conversational recommender systems; large language models (LLM)

ACM Reference Format:
Yaochen Zhu, Chao Wan, Harald Steck, Dawen Liang, Yesu Feng, Nathan Kallus, and Jundong Li. 2025. Collaborative Retrieval for Large Language Model-based Conversational Recommender Systems. In *Proceedings of the ACM Web Conference 2025 (WWW '25), April 28-May 2, 2025, Sydney, NSW, Australia.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3696410.3714908

*Work done when Yaochen and Chao were machine learning research interns at Netflix.

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.
WWW '25, Sydney, NSW, Australia
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1274-6/25/04
https://doi.org/10.1145/3696410.3714908

1 Introduction

Recommender systems (RS) have become an indispensable component on digital service platforms [16]. Traditional RSs, such as collaborative filtering [18], have demonstrated effectiveness in leveraging historical user-item interactions for recommendations. Conversational recommender systems (CRS) create a more engaging and interactive environment for users—they enable users to express preferences freely in natural language and refine their thoughts through multiple rounds of interactions [15, 31], where more precise and personalized recommendations can be offered to users.

Compared with traditional RSs, CRSs need to comprehensively consider both *items* and *context* in the dialogue, which is essential for user preference understanding and recommendation generations (see Fig. 1). Early CRSs [20, 31] used traditional RS models, such as factorization machines [27], and sequential models, such as recurrent neural networks (RNN) [6], to separately model items and context in the dialogue, where external item/word knowledge graphs (e.g., DBpedia [2] and ConceptNet [28]) are often leveraged to provide additional information. Subsequently, transformers [32] pretrained on external corpora (e.g., GPT-2 [24]) were introduced to enrich item/context representations with prior knowledge [8, 34, 42]. Meanwhile, semantic fusion strategies such as cross-attention [5, 42], mutual information maximization [34], and contrastive learning [43] were developed to integrate the representations of items and context to model context-aware user preferences.

Recently, large language models (LLMs), such as GPT-4o [23] and Claude 3.5-Sonnet [1], have demonstrated an unprecedented understanding of both items and context in natural language [40]. Pretrained on vast corpora across various domains, these LLMs can be viewed as unstructured knowledge databases that encompass extensive knowledge of items and their relations [26]. For instance, Xi et al. [39] showed that item knowledge prompted out from LLMs can enhance the recommendation accuracy of traditional RSs. Additionally, with their strong reasoning abilities, LLMs can generate

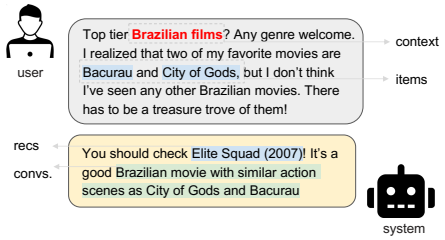


Figure 1: An example for conversational recommendations, with items and relevant context highlighted in user query.

more accurate recommendations by deriving user preferences based on better considerations of both items and context in the dialogue [35]. Built upon the recent advances in LLMs, He et al. [13] demonstrated that these powerful models (e.g., GPT-4o) can serve as good zero-shot CRSs, which substantially improve the recommendation performance compared with traditional CRS methods even though they are not directly trained on the CRS data.

While state-of-the-art LLMs possess extensive knowledge and reasoning abilities, they typically fall short in leveraging collaborative filtering (CF), a fundamental and effective technique in traditional RS [36, 41, 44]. The reason is that user-item interaction data are usually proprietary (and therefore not included in the LLMs' training corpora) and difficult to be fully described in natural language. Moreover, even if CF information can be integrated into LLM-based CRS, existing research indicates that adding more external knowledge does not necessarily enhance the LLMs (which are already very powerful) [10], as it can introduce noise that biases their behavior. Therefore, effectively utilizing CF information to complement the *context* in the dialogue and LLMs' inherent *content* knowledge presents a significant challenge for LLM-based CRS. As an aside, in a different line of work, CF has been used for improving LLMs in the classical recommendation setting [37]: Most works focus on *white-box* LLMs, where the model weights are accessible to the researcher [3, 14, 17, 41, 44]. White-box LLMs are generally smaller in scale compared to large proprietary LLMs, which are typically much more powerful, both in terms of their knowledge and reasoning capabilities. Due to the inaccessibility of model weights, however, combining CF with *black-box* LLMs is comparatively less explored [36, 38] (see Appendix A for more detailed discussions).

In this paper, to improve upon zero-shot LLMs, i.e., the current state-of-the-art [13], we propose CRAG, i.e., **C**ollaborative **R**etrieval **A**ugmented **G**eneration for LLM-based CRSs. To the best of our knowledge, CRAG is the first approach that combines state-of-the-art, black-box LLMs with collaborative filtering in the scenario of *conversational recommendations*, where context-aware CF knowledge can be introduced to enhance the recommendation performance. In our experiments in Sections 4.3 and 4.4, we show that CRAG leads to improved recommendation accuracy on two publicly available movie conversational recommendation datasets. We also provide several ablation studies to shed light on the inner workings of CRAG in Sections 4.5 and 4.6. Apart from that, we establish and release a refined version of the Reddit dataset [13] on movie recommendations, where the extraction of movies mentioned in the dialogues is substantially improved (see Section B). We also show (see **Finding 1** in Section B) that this improvement in extraction accuracy can have a considerable impact on the derived insights.

2 Problem Formulation

In this section, we formally define the CRS problem that we study in this paper. Let $\mathcal{U}$ denote the set of users and $\mathcal{I}$ the set of items. A conversation between a user and the CRS is denoted as $C = \{(u_t, s_t, \mathcal{I}_t)\}_{t=1}^{T}$, where at the t-th turn, $u_t \in \{\texttt{User}, \texttt{System}\}$ generates an utterance $s_t = (w_1, w_2, \ldots, w_{N_t})$, which is composed of N_t tokens from the vocabulary $\mathcal{V}$. $\mathcal{I}_t$ denotes the set of items mentioned in s_t. We assume that users can freely mention any item from $\mathcal{I}$ in the query, but the system can only recommend items from a fixed catalog (e.g., available movies on a specific platform like Netflix). We use $\mathcal{Q} \subseteq \mathcal{I}$ to denote the catalog of items available for recommendations. Here, we note that $\mathcal{I}_t$ is usually **not** annotated by the user and may be empty if no items are mentioned at the t-th turn. The CRS backbone is a black box LLM Φ, with historical interaction data $\mathbf{R} = \{0,1\}^{|\mathcal{U}_r| \times |\mathcal{I}|}$ available as an external collaborative filtering knowledge database. Users in $\mathcal{U}_r$ do not have to be the same as $\mathcal{U}$. $\mathbf{r}_{u \in \mathcal{U}_r} \in \{0,1\}^{|\mathcal{I}|}$ denotes the behavior patterns of user u, and is generally not included in the LLM training corpora.

The focus of this paper is mainly on the recommendation part of CRS, which aims to generate a ranked list of items $\hat{\mathcal{I}}_k$ from the catalog $\mathcal{Q}$ based on the current dialogue $C_{:k-1} = \{(u_t, s_t, \mathcal{I}_t)\}_{t=1}^{k-1}$ and the available interaction data $\mathbf{R}$, such that the generated $\hat{\mathcal{I}}_k$ best matches the groundtruth items in $\mathcal{I}_k$ (if $\mathcal{I}_k \neq \emptyset$ and $u_k = \texttt{System}$).

3 Approach

In this section, we introduce CRAG, a collaborative retrieval-augmented LLM for conversational recommendations. The overall framework of CRAG is illustrated in Fig. 2. CRAG consists of three components, ***(i)*** *LLM-based entity link*, ***(ii)*** *collaborative retrieval with context-aware reflection*, and ***(iii)*** *recommendation with reflect-and-rerank*, which will be outlined in the following parts.

3.1 LLM-based Entity Link

Entity linking, i.e., extracting items $\mathcal{I}_k$ from each utterance s_k and mapping them to the item database $\mathcal{I}$, is crucial for most CRS frameworks, as it bridges the gap between textual dialogues and external structured knowledge (e.g., knowledge graphs for traditional CRSs and interaction data $\mathbf{R}$ for CRAG). However, existing methods, e.g., Bayesian models [7] or supervised finetuning of transformers [13], rely on simulated data with seed items and struggle with handling abbreviations, typos, and ambiguity in item titles. Consequently, entity recognition noise is pervasive for the current CRS methods.

3.1.1 ***LLM-based Entity Extraction.*** In CRAG, we leverage the pretrained knowledge and reasoning ability of LLMs to extract mentioned items in each utterance s_t. Additionally, we analyze the *attitude* associated with each item to capture the sentiment or stance context under which the items are mentioned by the user in the dialogue. This process (Fig. 2-***(i)***) can be formally denoted as:

$$\mathcal{I}_t^{raw} = f_e\left(\Phi\left(T_e, F_e, s_t\right)\right). \tag{1}$$

Here, T_e is a task-specific prompt[1] that instructs the LLM Φ to reply with the standardized form of the items mentioned in the utterance s_t given that potential abbreviations, typos, and ambiguity could exist in s_t. In addition, to further improve the extraction efficiency,

[1]The details of the prompts defined in the main paper are provided in Appendix C.

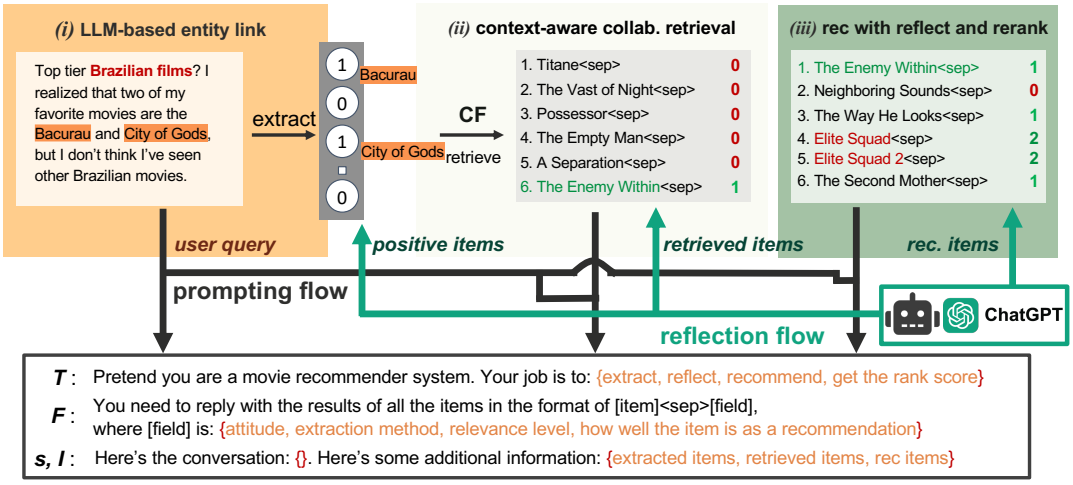


Figure 2: Overview of CRAG for CRS and its three components: *(i)* LLM-based entity link; *(ii)* context-aware collaborative retrieval, and *(iii)* recommendation with reflect and rerank. The reflection steps are emphasized in green arrows. The sub- and super-script for different task-specific prompt T, format instruction F, and item list $\mathcal{I}$ are omitted for simplicity.

we design a *batch inference format instruction* F_e to guide the LLM to reply with all the item-attitude pairs in the utterance s_t in the form of "[item]<sep>[attitude]", where we empirically set <sep> to "####" as the dummy tokens that separate the item name and the associated attitude in the response. In F_e, we explicitly instruct the LLM to output attitudes as numerical values in the range {-2, -1, 0, 1, 2}, representing attitude categories in the spectrum of {very negative, negative, neutral, positive, very positive}. This numerical encoding helps minimize errors in the generations. With F_e, the raw set of item-attitude pairs $\mathcal{I}_t^{raw} = \left\{\left(i_{t,j}^{raw}, a_{t,j}\right)\right\}_j$ can be trivially extracted from the LLM's output using a string processing function f_e that parses lines and the <sep> tokens.

3.1.2 ***Bi-level Match and Reflection.*** In the current stage, each raw item $i_{t,j}^{raw} \in \mathcal{I}_t^{raw}$ is a text string that may still be in non-standardized forms or contain small typos. To accurately link each raw item $i_{t,j}^{raw}$ to the item database $\mathcal{I}$, we introduce a bi-level match and reflection module that combines *character*-level and *word*-level fuzzy match with LLM-based reflection to post-fix the disagreements. Specifically, character-level match addresses typos in $i_{t,j}^{raw}$ [13], whereas word-level match links certain abbreviations (e.g., "Star Wars I") to their full names in the database $\mathcal{I}$ (e.g., "Star Wars I - The Phantom Menace"). We denote the two candidate sets produced by above match processes as $\mathcal{I}_t^{char}$, $\mathcal{I}_t^{word}$. Furthermore, we ask the LLM to reflect on the disagreements (if any) between $\mathcal{I}_t^{char}$ and $\mathcal{I}_t^{word}$, which is formally denoted as follows:

$$\mathcal{I}_t^{ref} = f_e^{ref}\left(\Phi\left(T_e^{ref}, F_e^{ref}, \mathcal{I}_t^{char}, \mathcal{I}_t^{word}, s_t\right)\right). \tag{2}$$

In this step, the task-specific prompt T_e^{ref} instructs the LLM to reflect on the differences between $\mathcal{I}_t^{char}$ and $\mathcal{I}_t^{word}$ based on the utterance s_t. In addition, the *batch reflection format instruction* F_e^{ref} guides the LLM to judge all the disagreements simultaneously and return the final reflection result of each item in the format of "[matched_item]<sep>[method]", where "[matched_item]" is the item that the LLM determines to be correctly linked to the database $\mathcal{I}$ (could be empty if none is found), and "[method]" in {char, word, both, none} indicates the correct matching strategy. Finally, the function f_e^{ref} processes the LLM's output by selecting, removing, or correcting each item based on the "[matched_item]" and "[method]" fields to form the final item set $\mathcal{I}_t^{ref}$ for s_t.

3.2 Context-Aware Collaborative Retrieval

After extracting and linking items for each utterance s_t in the dialogue $C_{:k-1}$ to the database $\mathcal{I}$, we introduce the collaborative retrieval module of CRAG. This module aims to retrieve context-relevant items based on the current dialogue $C_{:k-1}$ and historical interactions $\mathbf{R}$, which augments the prompt with collaborative filtering (CF) knowledge to enhance the LLM-based recommendations.

3.2.1 ***Collaborative Retrieval.*** Collaborative retrieval, similar to other retrieval-augmented generation (RAG) strategies [10], follows two main steps: query rewriting and similarity matching. The overall process for collaborative retrieval is defined as follows:

$$\mathcal{I}_k^{CR} = \mathrm{Top}_K(Sim\left(f_r\left(C_{:k-1}\right), \mathcal{Q}; \mathbf{R}\right)), \tag{3}$$

where the query rewrite function $f_r(C_{:k-1})$ aggregates the positively mentioned items from the current dialogue $C_{:k-1}$, i.e., $\mathcal{I}_k^q = \cup_{t=1}^{k-1}\mathcal{I}_t$, and converts them into a multi-hot variable $\mathbf{r}_k \in \{0,1\}^{|\mathcal{I}|}$. Since it is generally risky to extrapolate negatively mentioned items through CF (as the reason for disliking an item tends to be more subjective than collaborative) and because of the small number of negative item mentions in the dialogues (see Fig. 11 in the Appendix), we exclude these items from the collaborative retrieval model. Afterward, we retrieve the top-K items from the catalog $\mathcal{Q}$ based on their collaborative similarity (measured via the Sim function derived from the interaction data $\mathbf{R}$) with the items in $\mathcal{I}_k^q$.

Various CF methods [21, 22] can be used to learn the Sim function based on the interaction data $\mathbf{R}$. In this paper, we utilize a simple while effective adapted EASE [30] objective as follows:

$$\begin{aligned} \min_{\mathbf{W}} \quad & \|\mathbf{R}_{\mathcal{Q}} - \mathbf{R}\mathbf{W}\|_F^2 + \lambda \cdot \|\mathbf{W}\|_F^2 \\ \text{s.t. } & \mathbf{W}_{i,j} = 0, \forall i = \text{ReID}(j), \end{aligned} \tag{4}$$

where $\mathbf{R}_{\mathcal{Q}}$ selects the columns in $\mathbf{R}$ that correspond to the items in the catalog $\mathcal{Q}$, the asymmetric matrix $\mathbf{W} \in \mathbb{R}^{|\mathcal{I}| \times |\mathcal{Q}|}$ maps the space of items that users mention freely in the dialogue (i.e., $\mathcal{I}$) to the space of items available for recommendation in the catalog $\mathcal{Q}$, and the function ReID remaps the indices of the catalog items from $\mathcal{I}$ to $\mathcal{Q}$. The constraint in Eq. (4) prevents self-reconstruction from being used as a shortcut for the similarity matrix $\mathbf{W}$. Based on Eq. (4), the similarity function is then defined as $Sim(\mathcal{I}_k^q, \mathcal{Q}; \mathbf{R}) = \mathbf{r}_k^T \times \mathbf{W}$, which returns the similarity score of each item in $\mathcal{Q}$ relative to the positively mentioned items in $C_{:k-1}$. The scores are then used to select items in the collaborative retrieval $\mathcal{I}_k^{CR}$. In addition, $\mathbf{W}$ is adjusted by more recent item-popularities based on [29].

3.2.2 ***Context-Aware Reflection.*** Since the raw collaborative retrieval defined in Eq. (4) does not consider any context information in the current dialogue $C_{:k-1}$, directly augmenting the retrieved items $\mathcal{I}_k^{CR}$ in the prompt as extra collaborative knowledge could introduce context-irrelevant information, thereby biasing the LLM's recommendations. To address this issue, we post-process the retrieved items via an LLM-based context-aware reflection step as:

$$\mathcal{I}_k^{aug} = f^{aug}\left(\Phi\left(T^{aug}, F^{aug}, C_{:k-1}, \mathcal{I}_k^{CR}\right)\right), \tag{5}$$

where T^{aug} is the task-specific prompt that instructs the LLM to reflect on the contextual relevancy of items in $\mathcal{I}_k^{CR}$ based on the dialogue $C_{:k-1}$. In addition, F^{aug} is the *context-relevance batch reflection instruction* that guides the LLM to reply with the simultaneous judgment of all the items in $\mathcal{I}_k^{CR}$ in the format of `"[item]<sep>[relevance]"`, where `[relevance]` is a binary score in `{0, 1}` indicating whether or not a retrieved `[item]` is contextually relevant. After the reflection, only items that are judged as context-relevant are preserved in $\mathcal{I}_k^{aug}$, i.e., the context-aware collaborative retrieval, which is ready to be augmented into the prompt for recommendation generations. For example, in the example illustrated in Fig. 2-***(ii)***, although all the retrieved movies are similar to `City of God` and `Bacurau`, only `The Enemy Within` is Brazilian, where the rest are removed from $\mathcal{I}_k^{aug}$ after the reflection.

3.3 Recommendation with Reflect and Rerank

In this section, we discuss the *generation* phase of `CRAG`, which generates the final recommendation list with LLM based on the reflected collaborative retrieval $\mathcal{I}_k^{aug}$. This phase consists of three key steps: ***(i)*** collaborative query augmentation, ***(ii)*** LLM-based item generation, and ***(iii)*** reflect and rerank (post-processing).

3.3.1 ***Collaborative Query Augmentation.*** The preliminary step of utilizing the context-aware collaborative knowledge in $\mathcal{I}_k^{aug}$ is to augment it into the prompt for recommendations. This starts with adding a pretext to emphasize the collaborative nature of the retrieved items, such as: `"Below are items other users tend to interact with given the positive items mentioned in the dialogue:"`. Afterward, $\mathcal{I}_k^{aug}$ is transformed into a string, i.e., $I_{s,k}^{aug}$, that lists the similarity-ranked items separated by semicolons.

We note that $I_{s,k}^{aug}$ opens up to two interpretations in `CRAG`. From a RAG perspective, $I_{s,k}^{aug}$ serves as the extra CF information retrieved from an external user-item interaction database $\mathbf{R}$; from a recommendation perspective, $I_{s,k}^{aug}$ also represents the possible item candidates that could be used in the final recommendations. Based on these interpretations, we design two distinct prompts to instruct the LLM on how to use the augmented collaborative information: ***(i)*** a *rag* prompt that instructs the LLM to use the augmented information at its own discretion. ***(ii)*** a *rec* prompt that explicitly asks the LLM to consider the augmented items as candidates for recommendations (see Appendix C). Empirically, we find that different prompts work for different models. For example, GPT-4o enjoys the freedom in the *rag* prompt, whereas GPT-4 tends to ignore the retrieved items in $I_{s,k}^{aug}$ under the same prompt and instead needs the *rec* prompt to force it to consider the items in $I_{s,k}^{aug}$.

3.3.2 ***LLM-based Recommendations.*** After constructing the collaborative augmentation $I_{s,k}^{aug}$ from the item list $\mathcal{I}_k^{aug}$, it is appended to the current dialogue $C_{:k-1}$ and input into the LLM to generate a preliminary recommendation list. The collaborative augmented generation step in `CRAG` is formalized as follows:

$$\mathcal{I}_k^{rec} = f^{rec}\left(\Phi\left(T^{rec}, F^{rec}, C_{:k-1}, I_{s,k}^{aug}\right)\right), \tag{6}$$

where the prompt T^{rec} instructs the LLM to function as a CRS that generates a ranked item list as recommendations based on both the dialogue $C_{:k-1}$ and the collaborative augmentation $I_{s,k}^{aug}$. The format instruction F^{rec} guides the LLM to return the standardized item names seperated in lines. Eq. (6) takes into account both the dialogue context and the collaborative information to generate the recommendations, thereby addressing the key limitations of zero-shot LLM-based CRS: the lack of collaborative filtering abilities.

3.3.3 ***Reflect and Rerank.*** While the context-aware collaborative knowledge in $I_{s,k}^{aug}$ substantially enhances the relevancy of generated recommendations, it can also trigger a bias inherent in LLMs, where the attention mechanism tends to replicate the retrieved items in $I_{s,k}^{aug}$ at the beginning of the recommendations. Since items in $I_{s,k}^{aug}$ are retrieved by considering only the collaborative information (as the context-aware reflection in Eq. (5) only removes items), the most relevant items in $\mathcal{I}_k^{rec}$ generated by LLM (which are not necessarily in $I_{s,k}^{aug}$) may not be ranked on the top.

Here, a naive approach to mitigate the bias is to directly ask the LLM to rerank the items in the raw recommendations $\mathcal{I}_k^{rec}$. However, this may result in a nonsensical reranked list with missing items, probably due to the large semantic gap between the input items $\mathcal{I}_k^{rec}$ and the asked target, i.e., reranked items in $\mathcal{I}_k^{rec}$ based on the context relevancy. To bridge the gap, we propose a reflect-and-rerank module that asks the LLM to assign ordinal scores to each item in $\mathcal{I}_k^{rec}$ based on how well it aligns as a recommendation based on the dialogue $C_{:k-1}$. This can be formalized as:

$$\mathcal{I}_k^{r\&r} = f^{r\&r}\left(\Phi\left(T^{r\&r}, F^{r\&r}, C_{:k-1}, \mathcal{I}_k^{rec}\right)\right), \tag{7}$$

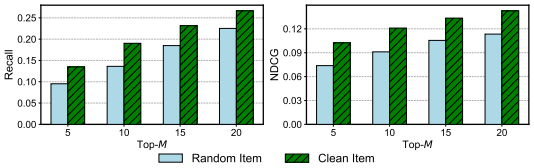


Figure 3: Comparison of zero-shot LLM on the Reddit-v2 dataset and the one with randomly replaced items.

where the task-specific prompt $T^{r\&r}$ instructs the LLM to reflect on the recommendations and assign scores to all the items in $\mathcal{I}_k^{rec}$ based on the dialogue $C_{:k-1}$. In addition, the *batch reflect-and-rerank instruction* $F^{r\&r}$ guides the LLM to return the scores for all the items in $\mathcal{I}_k^{rec}$ simultaneously in the format `"[item]<sep>[score]"`, where `[score]` ∈ `"{-2, -1, 0, 1, 2}"` corresponds to the level of recommendation quality in `{very bad, bad, neutral, good, very good}`. These scores serve as a reference for evaluating the relative suitability of each item, providing an intermediate step to address the semantic gap between the input item list $\mathcal{I}_k^{rec}$ and the context-aware reranked item list $\mathcal{I}_k^{r\&r}$. From the example in Fig. 2-***(iii)***, we can see that even though `The Enemy Within` is a good recommendation based on the collaborative information, after the reflect-and-rerank step of `CRAG`, more relevant ones such as `"Elite Squad"` and `"Elite Squad 2"` can be reranked on the top.

3.4 Conversations without Item Mentions

Finally, to introduce the context-aware CF information to improve the recommendations when user mentions no items in the dialogue $C_{:k-1}$, we first prompt the LLM to infer potential items the user might like based on $C_{:k-1}$. The generated items are then mapped to the database $\mathcal{I}$ via the strategy in Section 3.1, which can be treated as $\mathcal{I}_k^q$ in Eq. (3), and the remaining parts of `CRAG` remain the same.

4 Empirical Study

4.1 CRS Datasets

In this section, we introduce the established `Reddit-v2` dataset and the public `Redial` dataset used for CRS model evaluations.

4.1.1 ***Reddit-v2 Dataset.*** The largest real-world CRS dataset is the `Reddit` dataset established in [13], which consists of dialogues collected from the Reddit website under movie-seeking topics. In each dialogue, the movie-seeker is treated as the `user`, whereas the responder is treated as the `system`. In addition, items (i.e., movies) were extracted from the utterances using a T5 model [25] fine-tuned on a simulated utterance-item dataset. However, due to the limited capacity of both the T5 model and the simulated training data, this strategy suffered from rather low accuracy in the extracted movies.

To address the issue, we first refine the `Reddit` dataset (which we name ***Reddit-v2***) by extracting movies from the utterances with GPT-4o based on the strategy introduced in Section 3.1. To quantitatively verify the effectiveness of the movie name extraction, we reproduce the item-replacement experiment in [13], where we compare the performance of a `Zero-shot LLM` before and after randomly replacing the extracted movies in the `Reddit-v2` dialogues. The results are illustrated in Fig. 3. In Fig. 3 we can see a noticeable degradation in the recommendation performance when movies in the dialogues are randomly replaced (~0.05 for recall@5, where the difference of recall@5 is less than 0.01 in [13]). These results not only show that the refined `Reddit-v2` dataset is significantly cleaner, but also lead to **Finding 1**: *items mentioned in the conversations play a critical role for LLMs to generate recommendations.*

4.1.2 ***Redial Dataset.*** Another CRS dataset that we utilize in this paper is the `Redial` dataset, which is crowd-sourced from Amazon Mechanical Turk (AMT) by Li et al. [20] for movie recommendations. In the `Redial` dataset, movies mentioned in the utterances are tagged by the Turkers. Although this requirement eliminates the necessity of item recognition and database linking, it is not realistic in real-world applications. In addition, we found (as with He et al. [13]) that conversations in the `Redial` dataset can be overly polite, rigid, and succinct (e.g., replying with "Whatever Whatever I'm open to any suggestion." when being asked to clarify preferences), where the context information is comparatively insufficient.

4.2 Experimental Setup

For the `Reddit-v2` dataset, we use the subset of dialogues in the last month (i.e., *Dec. 2022*) as the test set, a subset from the prior month as the validation set, and all other dialogues as the training set (which are used for training traditional CRS baselines). Additionally, we establish the interaction data $\mathbf{R}$ based on the training dialogues, where each dialogue is treated as a pseudo-user i, and all the positively mentioned items are treated as the historical interactions $\mathbf{r}_i$. For both `Reddit-v2` and `Redial` datasets, we treat the set of mentioned items in all the dialogues as the item database $\mathcal{I}$ and all the items mentioned by the system as the catalog $\mathcal{Q}$. The statistics of both datasets are shown in Table 1 in the Appendix.

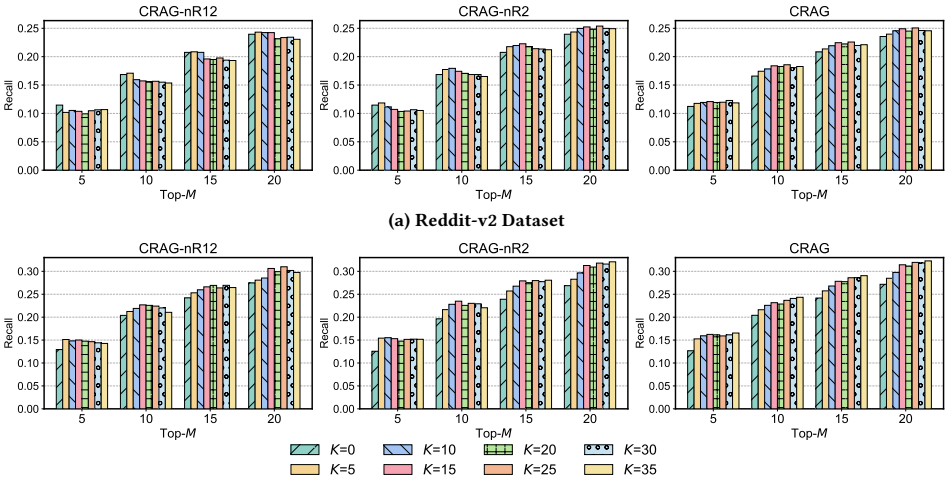
We consider two LLMs, i.e., GPT-4 and the latest GPT-4o, as the CRS backbone for `CRAG`[2]. We excluded other LLM models, such as GPT-3.5 and GPT-3.5-turbo, due to their significantly weaker instruction-following abilities. Here, we note that for the evaluation of LLM-based CRSs, the choice of LLM faces an inherent trade-off between item coverage and data-leakage risk. For GPT-4, approximately 15% of the movies in the item database $\mathcal{I}$ are released after its pretraining cut-off date, but all test dialogues are after its cut-off date. In contrast, GPT-4o covers all the items, but the test dialogues are before its cut-off date. However, even for GPT-4o, the risk of data leakage is low, as Reddit closed its crawling interface before GPT-4o. In addition, since the strongest baseline, i.e., the zero-shot LLM proposed in [13], will use the same LLM as `CRAG`, the comparison remains fair despite the trade-off in LLM selections.

4.3 Analysis of the Two-step Reflections

In our experiments, we first analyze the key contribution of `CRAG`, i.e., the two-step reflection process defined in Eqs. (5), (7), which improves the context-relevancy of collaborative retrieval and reranks the items in the recommendation list to prioritize more relevant ones. Specifically, we aim to explore when reflection works and how each reflection step contributes to the performance of `CRAG`.

4.3.1 ***Evaluation Setup.*** To answer the above research questions, we design two variants of `CRAG`, i.e., `CRAG-nR12`, `CRAG-nR2`, and explore their performance when the number of items in the raw collaborative retrieval (i.e., K in Eq. (3)) increases. Specifically, in

[2]The experimental results with GPT-4 backbone are provided in Appendix D.



(a) Reddit-v2 Dataset

(b) Redial Dataset

Figure 4: The influence of the number of items in the raw collaborative retrieval K (depicted in different color) on the recommendation performance of CRAG-nR12, CRAG-nR2, and CRAG. X-axis denotes the recall evaluated at top-M positions.

CRAG-nR12, we removed both reflection steps, whereas in CRAG-nR2, we remove only the final reflect-and-rerank step. We note that when $K = 0$, both CRAG-nR12 and CRAG-nR2 reduce to the zero-shot LLM as [13]. In addition, with the context-aware reflection module, the number of items actually getting augmented into the prompt could be less than K for both CRAG and CRAG-nR2. In the recommendation step, all three models are asked to recommend ***20*** movies.

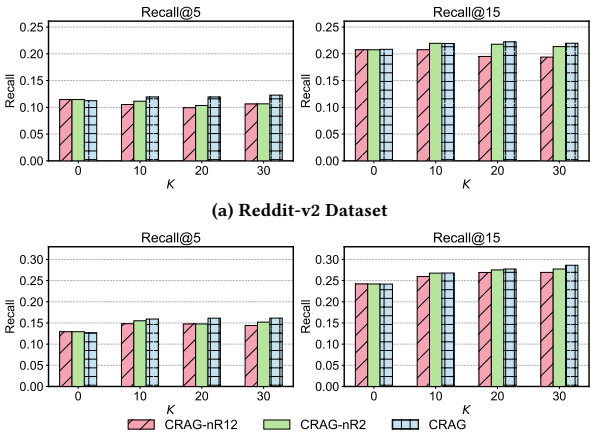
4.3.2 ***Intra-variant Comparisons.*** We first consider each CRAG variant *separately* and focus on the performance trend when the number of raw retrievals K increases. The results are shown in Fig. 4. The bar group at M shows the trend of recall@M when K increases from 0 to 35. In Fig. 4 we have three interesting findings:

Finding 2. *Naive collaborative retrieval is not very effective* (see the leftmost sub-figure). On the Reddit-v2 dataset, the performance of CRAG-nR12, i.e., the CRAG variant without any reflection, generally *decreases* when more items are retrieved and augmented into the prompt. This makes sense because the raw collaborative retrieval, i.e., $\mathcal{I}_k^{CR}$, does not consider any context in the dialogue, where context-irrelevant items bias the LLM's recommendations.

Finding 3. *Context-aware reflection improves the coverage of relevant items but struggles with the item rank* (see the middle sub-figure). This is reflected by the recall@20 bar group (as 20 is the number of movies we ask the LLM to recommend) for CRAG-nR2, where the metric now increases with a larger value of K compared with CRAG-nR12. However, recall@5, 10 of CRAG-nR2 quickly peak and then decrease as more items are retrieved. This suggests that, with K growing, an increased number of relevant items are recommended by CRAG-nR2, but they may not be ranked in top positions.

Finding 4. *Reflect-and-rerank addresses the rank bias and prioritizes most relevant items* (see the rightmost sub-figure). For CRAG with both reflection steps, we note that recall@5, 10 also increase with growing K (besides recall@20). This suggests that not only more relevant items are recommended by CRAG, but they are also increasingly getting ranked in the top (i.e., 5 and 10) positions.

The above findings lead to the conclusion that *LLMs are able to identify relevant items even if they cannot generate them.* This is exemplified by the fact that for $K > 0$, CRAG is able to exceed the performance of zero-shot LLMs (i.e., if $K = 0$) and include an increased number of relevant items in the top-M positions. As in CRAG, the LLM reflects on additional items generated by collaborative retrieval, this result suggests that the LLM is able to identify relevant items from among these additional items, even though the LLM was not able to generate these additional relevant items itself.



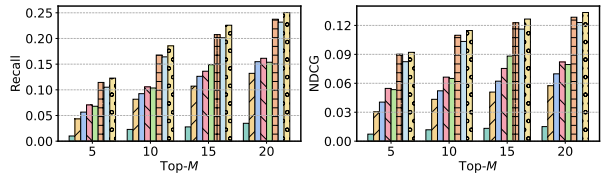
(a) Reddit-v2 Dataset

(b) Redial Dataset

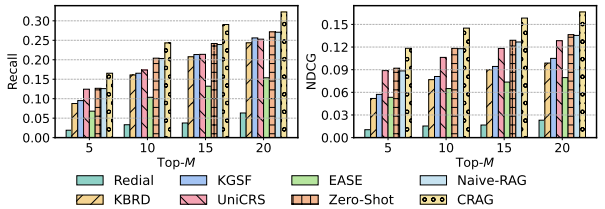
Figure 5: Comparison across different CRAG variants.

4.3.3 ***Cross-variant Comparison.*** In addition, we compare across the CRAG variants in Fig. 5, from which we can draw *two more interesting conclusions* that are not evident in Fig. 4:

Finding 5. *Self-reflection does not help.* We note that when $K = 0$, CRAG-nR12 and CRAG-nR2 degenerate to the zero-shot LLM, and CRAG degenerates to adding self-reflect and rerank on the zero-shot generations. The left-most bar group in Fig. 5 shows that when the recommendations are generated without external knowledge,

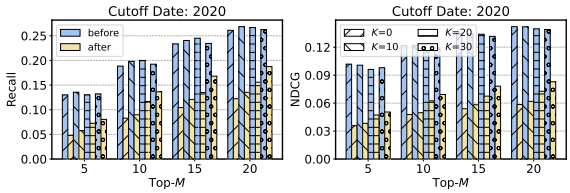


(a) Reddit-v2 Dataset

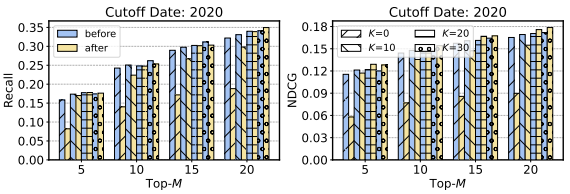


(b) Redial Dataset

Figure 6: Comparison between CRAG and various baselines on the established Reddit-v2 and Redial datasets.



(a) Reddit-v2 Dataset



(b) Redial Dataset

Figure 7: Comparison between the results on test dialogues separated by the release year of the groundtruth movies.

self-reflection on the final recommendation list does not help. This makes sense, as for the `zero-shot LLM` model, the items reflected upon are generated based only on the same LLM's internal knowledge, where the self-reflection cannot introduce new knowledge.

Finding 6. *Context is important for both reflection steps.* The larger improvement of `CRAG` over `CRAG-nR12` and `CRAG-nR2` on the `Reddit-v2` dataset compared with the `Redial` dataset shows that the two-step reflection works better for dialogues with richer context information (as for the `Reddit-v2` dataset). This shows that `CRAG` can effectively combine collaborative retrieval with the context-understanding ability of LLMs to improve LLM-based CRS.

4.4 Comparison to Baselines

In this section, we compare `CRAG` with various state-of-the-art RNN-, transformer-, and LLM-based CRS baselines as follows:

- **Redial** [20] leverages a denoising autoencoder to model the mentioned items and to generate recommendations, while an RNN is used to model and generate conversations.
- **KBRD** [5] introduces a relational GNN (RGNN) on the DBpedia knowledge graph (KG) to model entities, and optimize similarity between co-occurring words and entities to fuse the semantics.
- **KGSF** [42] incorporates a word-level KG from ConceptNet to model the conversations and use mutual information maximization w.r.t. entity KG embeddings to fuse the entity information.
- **UniCRS** [34] introduces a pretrained transformer to capture the context information, with cross-attention [32] w.r.t. the entity KG embeddings (RGNN) used for semantic fusion.
- **Zero-shot LLM** [13] directly inputs the dialogue with task-specific prompt and format instruction for CRS without any information retrieval from external knowledge databases.
- **Naive-RAG** [19] retrieves item-related sentences from a database of movie plots and metadata based on the query-sentence semantic similarity to augment the zero-shot LLM.

For `Redial`, `KBRD`, and `KGSF`, we follow the implementation in CRSLab, where we adapt the evaluation codes (which replicate each conversation multiple times such that each has exactly one groundtruth) to make it consistent with the evaluation of `CRAG`. In addition, we limit the recommendations of all the baselines to the items in the catalog $\mathcal{Q}$. Finally, we include **EASE** [30] as a non-CRS baseline (as we adapt it for collaborative retrieval), whose recommendations are based only on items mentioned in the dialogue.

The comparisons are shown in Fig. 6, where we can see that the `Redial` model, which separately models and generates items and conversations, achieves the lowest performance. `KBRD` and `KGSF` improve over `redial` by introducing an external KG on the entities and strategies to fuse the entity and context semantics in the dialogue. `UniCRS` further leverages a pretrained transformer to model the context, which achieves the best performance among all the non-LLM-based baselines. However, due to the vast knowledge and reasoning ability of modern LLMs, Fig. 6 shows that a `Zero-shot LLM` improves substantially over the traditional methods. Regarding the RAG-based methods, we have the following findings:

Finding 7. Interestingly, we find that, `Naive-RAG`, which augments the `Zero-shot LLM` by retrieving relevant content/metadata as documents into the prompt, actually *degrades* in performance. The reason could be the large semantic gap between words in the conversations and the implicit user preference. For example, for the dialogue in Fig. 2, most documents retrieved by `Naive-RAG` are from movies that directly have Brazil/Brazilian in the title, but the user mentioned Brazilian only as a quantifier to his/her true preferences, i.e., movies similar to `City of God` and `Bacurau`.

Finding 8. `CRAG` achieves the best performance by all the metrics across both datasets compared with both `Zero-Shot LLM` and `Naive-RAG`, which further demonstrates the effectiveness of the collaborative retrieval with two-step reflection in `CRAG`.

4.5 Evaluation w.r.t. the Recency of Items

In this section, we shed more light on the main effect that we identified for `CRAG`: while `CRAG` improves the recommendation accuracy for all cases, the gains are more substantial for movies that were released more recently. This is corroborated by the following experiment: We first select a cut-off year (e.g., `2020`, but other years generally lead to similar results) and split the test dialogues into `before` and `after` groups: in the `before`-group, all groundtruth

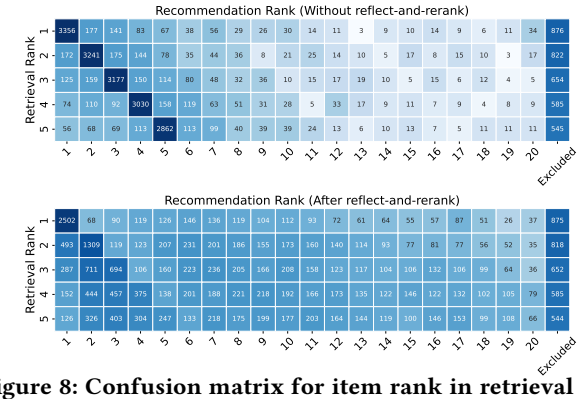


Figure 8: Confusion matrix for item rank in retrieval and recommendation w, w/o the reflect-and-rerank step.

movies are released before the cut-off year, whereas in the `after`-group, at least one movie is released after the cut-off year. The results in Fig. 7 show the following interesting findings:

Finding 9. *LLMs are less effective in recommending more recent items.* This is reflected by the overall lower performance of CRAG on the `after`-group (yellow bars) than the `before`-group (blue bars).

Finding 10. CRAG *leads to larger improvements for the recommendation of more recent items.* This is reflected by the larger metric increases for CRAG on the `after`-group with the growing of K, i.e., the number of items in the raw collaborative retrieval, compared to the `before`-group. Visually, this is reflected by the steeper metric improvements of the yellow bars (i.e., `after`-group) compared to the blue bars (i.e., `before`-group) when K increases in Fig. 7.

4.6 Retrieval and Recommendation

Finally, we point out the importance of reflect-and-rerank step. To this end, we examine the relation between items in the context-aware collaborative retrieval $\mathcal{I}_k^{aug}$ that get augmented into the prompt and the final list of items getting recommended by `CRAG-nR2` (i.e., $\mathcal{I}_k^{rec}$ in Eq. (6)) and CRAG (i.e., $\mathcal{I}_k^{r\&r}$ in Eq. (7)). Fig. 8 shows the confusion matrix, where the element at row i and column j denotes the number of times when the j-th item in $\mathcal{I}_k^{aug}$ is put in the i-th position of recommendations (to save space, we selected $K = 20$ and show only top 5 rows). The matrix at the top of Fig. 8 shows the results of `CRAG-nR2`, which leads to the following findings:

Finding 11. *LLMs have the bias to replicate the retrieved items.* The dominating diagonal elements in the confusion matrix at the top of Fig. 8 show that LLMs are indeed biased to replicating retrieved items at the beginning of the recommendation list. This is undesirable as the retrieved items only consider the CF information (as context-aware reflection only removes context-irrelevant items).

Finding 12. *LLMs tend to replace items in the collaborative retrieval in place instead of removing them and filling in the next ones.* Otherwise, given the large number of items excluded from recommendations, the confusion matrix at the top of Fig. 8 should have larger values for all lower-triangular elements (which denote the cases of upwardly lifted items from the retrieval $\mathcal{I}_k^{aug}$ to the recommendation list $\mathcal{I}_k^{rec}$) instead of only dominant diagonal elements.

With reflect-and-rerank introduced on the final recommendation list, the dominating diagonal elements vanish in the confusion-matrix for CRAG at the bottom of Fig. 8, which indicates that more relevant items are prioritized at the top of the recommendation list $\mathcal{I}_k^{r\&r}$, irrespective of whether these items were from collaborative retrieval $\mathcal{I}_k^{aug}$ or new ones generated by the LLM outside $\mathcal{I}_k^{aug}$.

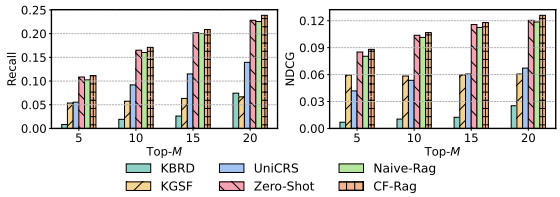


Figure 9: Comparison of CRAG to the baselines on the conversations with no item mentions in the Reddit-v2 dataset.

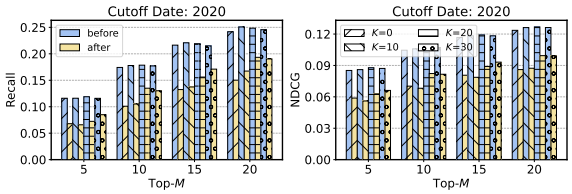


Figure 10: Results on Reddit-v2 where conversations are separated by the release year of the movies to be recommended.

4.7 Conversations without Items Mentions

In this section, we provide the results for CRAG on conversations with no explicitly mentioned items based on the pre-generation trick introduced in Section 3.4. We focus on the `Reddit-v2` dataset, as almost all the conversations in the `Redial` dataset contain at least one movie being mentioned. The comparison with baselines in Fig. 9 leads to **Finding 13**: *The relative performance of the baselines generally shows a similar trend as the cases where there are items mentioned in the conversation (see Fig. 6)*, although the improvement of CRAG over the `Zero-shot LLM` baseline is not as substantial.

Analogous to the experiments in Fig. 7, the results for the conversations with no movie mentions are shown in Fig. 10, which lead to **Finding 14**: Despite the smaller overall improvement of CRAG over `Zero-shot LLM` in the case where no items are mentioned in the dialogue, *the improvement in the recommendation of movies with more recent release-years is still very evident.* This again demonstrates the increased effectiveness of CRAG over `zero-shot LLMs` in recommending movies that are more recently released.

5 Conclusions

In this paper, we proposed CRAG, the first approach that combines state-of-the-art, black-box LLMs with collaborative filtering for CRS. In our experiments, we showed that this results in improved recommendation accuracy on two publicly available movie conversational recommendation datasets, eclipsing the current state-of-the-art CRS methods, i.e., the zero-shot LLMs. We also provided several ablation studies to shed light on the inner workings of this approach. In particular, we found that the recently released movies benefited especially from CRAG. Apart from that, we established a refined version of the Reddit dataset on movie recommendations, where the extraction of movies mentioned in the dialogues is greatly improved. We also showed that this improvement in movie extraction accuracy can have a considerable impact on the derived insights.

References

[1] Anthropic. 2024. Claude 3.5 Sonnet. https://www.anthropic.com/news/claude-3-5-sonnet.

[2] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. 2007. Dbpedia: A nucleus for a web of open data. In *International Semantic Web Conference*. Springer, 722–735.

[3] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *RecSys*. 1007–1014.

[4] Mohamed Ishmael Belghazi, Aristide Baratin, Sai Rajeshwar, Sherjil Ozair, Yoshua Bengio, Aaron Courville, and Devon Hjelm. 2018. Mutual information neural estimation. In *ICML*. 531–540.

[5] Qibin Chen, Junyang Lin, Yichang Zhang, Ming Ding, Yukuo Cen, Hongxia Yang, and Jie Tang. 2019. Towards Knowledge-Based Recommender Dialog System. In *EMNLP*.

[6] Junyoung Chung, Caglar Gulcehre, Kyunghyun Cho, and Yoshua Bengio. 2014. Empirical evaluation of gated recurrent neural networks on sequence modeling. In *NeurIPS Workshop on Deep Learning*.

[7] Joachim Daiber, Max Jakob, Chris Hokamp, and Pablo N Mendes. 2013. Improving efficiency and accuracy in multilingual entity extraction. In *Semantic*. 121–124.

[8] Yue Feng, Shuchang Liu, Zhenghai Xue, Qingpeng Cai, Lantao Hu, Peng Jiang, Kun Gai, and Fei Sun. 2023. A large language model enhanced conversational recommender system. *arXiv preprint arXiv:2308.06212* (2023).

[9] Chongming Gao, Wenqiang Lei, Xiangnan He, Maarten de Rijke, and Tat-Seng Chua. 2021. Advances and challenges in conversational recommender systems: A survey. *AI Open* 2 (2021), 100–126.

[10] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023).

[11] J Gu, Z Lu, H Li, and VOK Li. 2016. Incorporating copying mechanism in sequence-to-sequence learning. In *ACL*.

[12] Donghoon Ham, Jeong-Gwan Lee, Youngsoo Jang, and Kee-Eung Kim. 2020. End-to-end neural pipeline for goal-oriented dialogue systems using GPT-2. In *ACL*. 583–592.

[13] Zhankui He, Zhouhang Xie, Rahul Jha, Harald Steck, Dawen Liang, Yesu Feng, Bodhisattwa Prasad Majumder, Nathan Kallus, and Julian McAuley. 2023. Large language models as zero-shot conversational recommenders. In *CIKM*.

[14] Wenyue Hua, Shuyuan Xu, Yingqiang Ge, and Yongfeng Zhang. 2023. How to index item ids for recommendation foundation models. In *SIGIR*. 195–204.

[15] Dietmar Jannach, Ahtsham Manzoor, Wanling Cai, and Li Chen. 2021. A survey on conversational recommender systems. *ACM Computing Surveys (CSUR)* 54, 5 (2021), 1–36.

[16] Dietmar Jannach, Markus Zanker, Alexander Felfernig, and Gerhard Friedrich. 2010. *Recommender Systems: An Introduction*. Cambridge University Press.

[17] Sein Kim, Hongseok Kang, Seungyoon Choi, Donghyun Kim, Minchul Yang, and Chanyoung Park. 2024. Large language models meet collaborative filtering: an efficient all-round LLM-based recommender system. In *KDD*. 1395–1406.

[18] Yehuda Koren, Steffen Rendle, and Robert Bell. 2021. Advances in collaborative filtering. *Recommender Systems Handbook* (2021), 91–142.

[19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*. 9459–9474.

[20] Raymond Li, Samira Ebrahimi Kahou, Hannes Schulz, Vincent Michalski, Laurent Charlin, and Chris Pal. 2018. Towards deep conversational recommendations. In *NeurIPS*.

[21] Dawen Liang, Rahul G Krishnan, Matthew D Hoffman, and Tony Jebara. 2018. Variational autoencoders for collaborative filtering. In *WWW*. 689–698.

[22] Andriy Mnih and Russ R Salakhutdinov. 2007. Probabilistic matrix factorization. In *NeurIPS*, Vol. 20.

[23] OpenAI. 2024. Hello GPT-4o. https://openai.com/index/hello-gpt-4o/.

[24] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[25] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR* 21, 140 (2020), 1–67.

[26] Xubin Ren, Wei Wei, Lianghao Xia, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2024. Representation learning with large language models for recommendation. In *WWW*. 3464–3475.

[27] Steffen Rendle. 2010. Factorization machines. In *ICDM*. 995–1000.

[28] Robyn Speer, Joshua Chin, and Catherine Havasi. 2017. Conceptnet 5.5: An open multilingual graph of general knowledge. In *AAAI*, Vol. 31.

[29] Harald Steck. 2019. Collaborative Filtering via High-Dimensional Regression. *arXiv preprint arXiv:1904.13033* (2019).

[30] Harald Steck. 2019. Embarrassingly shallow autoencoders for sparse data. In *WWW*. 3251–3257.

[31] Yueming Sun and Yi Zhang. 2018. Conversational recommender system. In *SIGIR*. 235–244.

[32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, and Aidan N Gomez. 2017. Attention is all you need. In *NeurIPS*.

[33] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. 2008. Extracting and composing robust features with denoising autoencoders. In *ICML*. 1096–1103.

[34] Xiaolei Wang, Kun Zhou, Ji-Rong Wen, and Wayne Xin Zhao. 2022. Towards unified conversational recommender systems via knowledge-enhanced prompt learning. In *KDD*. 1929–1937.

[35] Wei Wei, Xubin Ren, Jiabin Tang, Qinyong Wang, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2024. LLMRec: Large language models with graph augmentation for recommendation. In *WSDM*. 806–815.

[36] Junda Wu, Cheng-Chun Chang, Tong Yu, Zhankui He, Jianing Wang, Yupeng Hou, and Julian McAuley. 2024. CoRAL: Collaborative Retrieval-Augmented Large Language Models Improve Long-tail Recommendation. In *KDD*. 3391–3401.

[37] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, et al. 2024. A survey on large language models for recommendation. *World Wide Web* 27, 5 (2024), 60.

[38] Yunjia Xi, Weiwen Liu, Jianghao Lin, Xiaoling Cai, Hong Zhu, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, and Yong Yu. 2024. Towards open-world recommendation with knowledge augmentation from large language models. In *RecSys*. 12–22.

[39] Yunjia Xi, Weiwen Liu, Jianghao Lin, Xiaoling Cai, Hong Zhu, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, Rui Zhang, et al. 2024. Towards open-world recommendation with knowledge augmentation from large language models. In *RecSys*.

[40] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2024. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2024).

[41] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *ICDE*. 1435–1448.

[42] Kun Zhou, Wayne Xin Zhao, Shuqing Bian, Yuanhang Zhou, Ji-Rong Wen, and Jingsong Yu. 2020. Improving conversational recommender systems via knowledge graph based semantic fusion. In *KDD*. 1006–1014.

[43] Yuanhang Zhou, Kun Zhou, Wayne Xin Zhao, Cheng Wang, Peng Jiang, and He Hu. 2022. C^2-CRS: Coarse-to-fine contrastive learning for conversational recommender system. In *WSDM*. 1488–1496.

[44] Yaochen Zhu, Liang Wu, Qi Guo, Liangjie Hong, and Jundong Li. 2024. Collaborative large language model for recommender systems. In *WWW*. 3162–3172.

Appendix

In the appendix, we discuss the related work of CRAG (see Section A), provide detailed analysis and statistics of the established Reddit-v2 dataset (see Section B), provide the details of the prompts used in the main paper (see Section C), and provide additional experimental results of CRAG with GPT-4 backbone (see Section D).

A Related Work

In this section, we review the related work of CRAG, which includes both conversational recommender systems and research on large language models (LLM) with collaborative filtering.

A.1 Conversational Recommender Systems

Conversational recommender systems (CRS) aim to generate recommendations through natural language interactions with users [9, 15]. Throughout the dialogue with users, there usually exist two types of information, i.e., *item* and *context*, where the latter denotes the non-item words that users utter alongside the items to express their preferences. To handle these two aspects, CRS models generally contain the following two phases: ***(i)*** *modeling*, which learns to understand both items and context mentioned in the dialogue, and ***(ii)*** *generation*, which generates items and words in natural language based on the dialogue understanding as the response.

From the *modeling* perspective, a typical CRS involves three key components: entity modeling, context modeling, and semantic fusion. Various traditional recommendation models, such as factorization machines [27] and denoising autoencoders [33], have been used to model the items mentioned in the dialogues [5, 20]. Context modeling, on the other hand, utilizes language models with recurrent neural networks (RNNs) [6] or transformers [12, 32] to capture the conversational flow and background information. To integrate item and context information, semantic fusion techniques such as mutual information maximization [4] and cross-attention mechanisms [32] have been employed [34, 42] to comprehensively understand the user preferences. In addition, knowledge databases, such as DBPedia [2] and ConceptNet [28], have been used to enhance both item and context modeling with external information.

For the *generation* phase, early methods introduced a switch mechanism, i.e., a binary predictor, to decide whether the next token to be generated should be a word or an item [5, 20]. Afterward, approaches such as copy mechanism [11] are used to align item and word tokens in the same generation space. Recently, Wang et al. [34] introduced an `<item>` token when generating the context, which enables the system to comprehensively consider the generated context for item recommendations. *The advent of large language models (LLMs) has further blurred the boundaries between items and context, as well as between modeling and generation phases of CRSs.* LLMs possess extensive knowledge and reasoning abilities, allowing them to understand items and context simultaneously in the form of natural language. Moreover, the generation of items and context in the responses can be unified in the textual space, leveraging the LLM's capacity to produce coherent natural language outputs.

However, LLMs are comparatively less effective in the recommendation of more recent items due to fewer relevant documents in the training corpora. In addition, LLMs struggle to leverage collaborative filtering knowledge, which is highly informative for recommendations. These two challenges motivate us to introduce collaborative retrieval with two-step reflection in CRAG to augment the LLM's recommendations with context-aware CF knowledge.

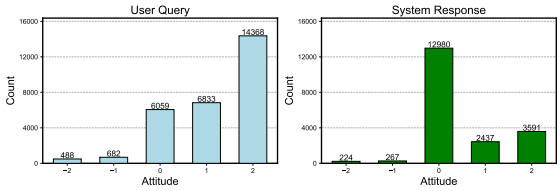


Figure 11: Distribution of attitudes for movie mentions in user queries and system responses for the Reddit-v2 test set.

A.2 LLM with Collaborative Filtering

Recently, recommender system researchers have recognized the importance of integrating collaborative filtering (CF) with large language models (LLMs) to further enhance their recommendation abilities [37]. Most works focus on the *white-box* LLMs, where the model weights are accessible to the researchers. One promising strategy is to introduce new tokens for users/items to capture the collaborative filtering knowledge. These tokens can be independently assigned for each user/item [3, 44] or clustered based on semantic indexing [14]. The embeddings associated with the user/item tokens can be learned with language modeling on natural language sequences converted from user-item interactions [44] or predicted from pretrained CF models based on external neural networks [17, 41]. While white-box LLMs provide the possibility to introduce CF knowledge through model finetuning, they are generally smaller in scale compared to large proprietary LLMs. Due to the inaccessibility of model weights, combining CF with *black-box* LLMs is less explored. One strategy is to augment CF models with LLMs' analysis of user preferences [26, 35, 38]. To use the LLM itself as the recommender, Wu et al. [36] proposed to transform user-item interactions into the prompt for LLMs to understand the user preference and utilize a policy network to reduce the redundancy. However, these approaches focus on traditional symmetric CF settings, which are not suitable for CRS with asymmetric item mention/recommendation and complex contextual information.

CRAG distinguishes itself by effectively combining CF with black-box, state-of-the-art LLMs that comprehensively consider the interaction data and the context in the dialogue for recommendations. By introducing context-aware retrieval and a two-step reflection process, CRAG addresses the limitations of zero-shot LLM-based CRS and substantially enhances the recommendation quality.

B Statistics of the Reddit-v2 Dataset

To address item extraction noise in the original Reddit dataset [13], we run the LLM-based entity extraction strategy introduced in Section 3.1 to establish the refined Reddit-v2 dataset. For evaluation, we select the subset of dialogues with start date on the last month (i.e., *Dec. 2022*) as the test set, where the meta information (as with the Redial dataset) is illustrated in Table 1. In addition, the distribution of attitudes for user query and system response is illustrated in Fig. 11. The large number of attitudes 0 for system response is due to succinct recommendations with only movie names, where

Table 1: Statistics of the Reddit-v2 and Redial test sets, where #Conv. denotes the number of test samples with items mentioned in the dialogue, and #Conv. (X) denotes the number of test samples with NO items mentioned in the dialogue.

Dataset	#Conv.	#Conv. (X)	#Items	#Catalog
Reddit-v2	5,613	2,231	5,384	4,752
Redial [20]	2,998	619	1,915	1,476

the attitude is difficult to judge by LLM. Therefore, 0 is also treated as a positive attitude for the system responses.

C Prompts Used in the Main Paper

In this section, we provide the task-specific prompts and format instructions that we defined in the main paper for the prompting and reflection process of CRAG (see Fig. 2) as follows:

Eq. (1): LLM-based Entity Extraction

T_e: Pretend you are a movie recommender system. You (a recommender system) will be given a user's query that seeks movie recommendations. Based on the query, you need to extract movie names mentioned in the user's query and analyze the user's attitude toward each movie. You need to reply with standardized movie names (with grammatical errors corrected and abbreviations fixed), as well as the user's attitude toward the movie.

F_e: Specifically, the movie names need to be formatted in the IMDB style, with the year bracketed if possible (do not add the year if you are not sure). In addition, the attitude is represented in one of [-2, -1, 0, 1, 2], where -2 stands for very negative, -1 stands for negative, 0 stands for neutral, 1 stands for positive, and 2 stands for very positive. You need to reply with the number as an attitude instead of the textual description. If there are movie names mentioned in the query, list each movie name and the user's attitude (number in -2 to 2) in the form of movie_name####attitude, where different movies are listed in different lines with no extra sentences. Reply NO if no movie names are mentioned in the query.

s_t: Here is the user's query: {}.

Eq. (2): Reflection on Bi-level Matched Entities

T_e^{ref}: Pretend you are a movie recommender system. You, as the recommender system, will be given part of the dialogue between a user seeking a movie recommendation and yourself, along with the extracted movie names (which may potentially be incorrect). Even if the extracted movie names are correct, the wording might not be precise. Therefore, you will be provided with the best match for each extracted movie name from an external database using (1) character-level fuzzy match and (2) word-level BM25 match (a space will be provided if no name can be found via the word-level match). Often, since these two matching methods focus on different levels of granularity, their results may not align. Based on the results, you must determine whether each movie name extraction is correct and what the precise movie name for that extracted name should be from the database.

F_e^{ref}: To reflect on this, for each extracted movie, you must respond with three terms separated by ####: (1) the raw movie name mentioned in the dialogue (raw refers to the exact text from the dialogue), (2) the precise movie name selected from fuzzy match or BM25 (reply with a space if the movie name extraction is incorrect or if neither match is precise), and (3) the correct extraction method, choosing from [fuzzy, BM25, none, both]. If the fuzzy match and BM25 results differ but both are probable, select the more probable one based on context as the correct name. List the reflection on each movie name in the exact form of raw _name####correct_name####method on a new line with no additional terms or sentences.

s_t: Here is the user's query: {}.

$\mathcal{I}_t^{char}, \mathcal{I}_t^{word}$: Here are extracted movie names, fuzzy matches, and BM25 matches from the movie database in the form of extracted_name####fuzzy_match####BM25_match: {}.

Eq. (5): Reflection on the Collaborative Retrieval

T^{aug}: Pretend you are a movie recommender system. I will give you a conversation between a user and you (a recommender system), as well as movies retrieved from the movie database based on the similarity with movies mentioned by the user in the context. You need to judge whether each retrieved movie is a good recommendation based on the context.

F^{aug}: You need to reply with the judgment of each movie in a line, in the form of movie_name####judgment, where judgment is a binary number 0, 1. Judgment 0 means the movie is a bad recommendation, whereas judgment 1 means the movie is a good recommendation.

$C_{:k-1}$: Here is the conversation: {}.

$\mathcal{I}_k^{CR}$: Here are retrieved movies: {}.

Eq. (6): LLM-based Recommendations

T^{rec}: Pretend you are a movie recommender system. I will give you a conversation between a user and you (a recommender system). Based on the conversation, you need to reply with 20 movie recommendations without extra sentences.

F^{rec}: List the standardized title of each movie on a separate line.

$C_{:k-1}$: Here is the conversation: {}.

$\mathcal{I}_{s,k}^{aug}$: Based on movies mentioned in the conversation, here are some movies that are usually liked by other users: {}.

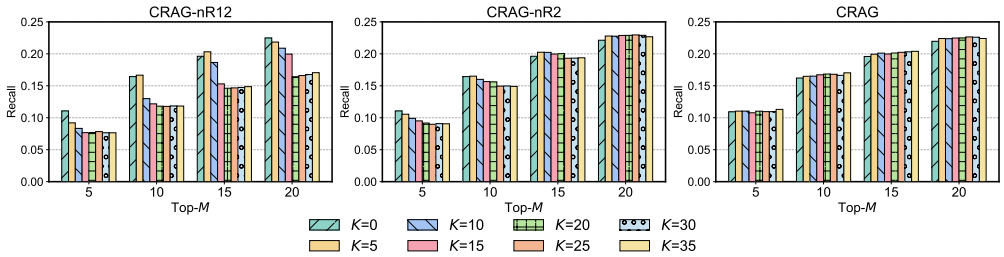


Figure 12: The influence of the number of items in the raw collaborative retrieval K (depicted by bars with different colors) on the recommendation performance of CRAG-nR12, CRAG-nR2, and CRAG (all with GPT-4 backbone). X-axis denotes the recall evaluated at top-M positions.

rag prompt (GPT-4o): Use the above information at your discretion (i.e., do not confine your recommendation to the above movies).
rec prompt (GPT-4): Consider using the above movies for recommendations."

Eq. (7): Reflect and Rerank

$T^{r\&r}$: Pretend you are a movie recommender system. I will give you a conversation between a user and you (a recommender system), as well as some movie candidates from our movie database. You need to rate each retrieved movie as recommendations into five levels based on the conversation: 2 (great), 1 (good), 0 (normal), -1 (not good), -2 (bad).

$F^{r\&r}$: You need to reply with the rating of each movie in a line, in the form of movie_name####rating, where the rating should be an Integer, and 2 means great, 1 means good, 0 means normal, -1 means not good, and -2 means bad.

$C_{:k-1}$: Here is the conversation: {}.

$\mathcal{I}_k^{rec}$: Here are the movie candidates: {}.

D Experimental Results on GPT-4 Backbone

In this subsection, we provide the experimental results of CRAG with GPT-4 backbone on the **Reddit-v2** dataset. Please note that when generating collaborative retrieval augmented recommendations with Eq. (6), we use the *rec* prompt instead of the *rag* prompt.

D.1 Analysis of the Two-step Reflections

We first run the experiments of CRAG with GPT-4 backbone under the setting of Section 4.3 to explore the effects of the two reflection processes in CRAG on the model performance. The results are summarized in Fig. 12. From the figure we can find that the three CRAG variants follow the same trend with the case with GPT-4o backbone, where ***(i)*** naive collaborative retrieval in CRAG-nR12 hurts the performance due to the introduction of context-irrelevant collaborative information, ***(ii)*** after introducing the context-aware reflection step, CRAG-nR2 improves the item coverage compared with CRAG-nR12 but it still struggles with item rank, and ***(iii)*** by adding reflect-and-rerank on top of CRAG-nR2, the proposed CRAG leads to the prioritization of more relevant items at the top positions.

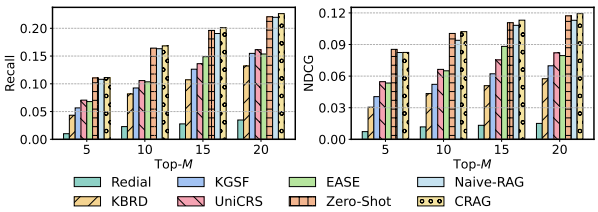


Figure 13: Comparison of CRAG (with GPT-4 backbone) to the baselines on the conversations with no item mentions in the Reddit-v2 dataset.

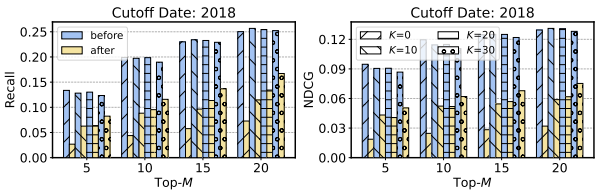


Figure 14: Results of CRAG (with GPT-4 backbone) on the Reddit-v2 dataset where the conversations are separated by the release year of the movies to be recommended.

D.2 Comparison with Baselines

We then compare the CRAG with GPT-4 backbone with the baselines introduced in Section 4.4 of the main paper. The results are illustrated in Fig. 13, where the relative performance among the methods remains the same as the results of CRAG with GPT-4o backbone illustrated in Fig. 6. In addition, CRAG with GPT-4 backbone also achieves the best performance compared with all the baselines.

D.3 Evaluation w.r.t. the Recency of Items

Finally, we evaluate the performance of CRAG with GPT-4 backbone w.r.t. the item recency. Since the cut-off date of GPT-4 is two years before that of GPT-4o, we set the cut-off year (to split the test data) to 2018, which is also two years prior to the cut-off year used in Fig. 7 for GPT-4o backbone, and evaluate the model on the before and after groups with the same setting as Section 4.5 in the main paper. The results are summarized in Fig. 14. From Fig. 14 we can come to the same conclusion that the improvement for CRAG with GPT-4 backbone over the zero-shot LLM model is largely due to the increased accuracy in recommendations of more recent items.